



Project Report

DSA

Bus Reservation System

SUBMITTED TO
Engr. Faisal Zia

Presented by:
Maheen Akbar 09-131242-041
Kinza Batool 09-131242-038

1. Introduction

BusRes is a fully-featured desktop bus seat reservation system developed in Python using the tkinter GUI framework and SQL Server Express as the backend database. The system serves three distinct user roles — Administrator, Manager, and Passenger — each with a dedicated dashboard, role-specific features, and controlled access to data.

The project serves as a practical demonstration of Data Structures and Algorithms (DSA) concepts applied in a real-world application context, covering linear and non-linear data structures, sorting algorithm comparison, multiple searching strategies, and graph-based pathfinding.

1.1 Project Objectives

- Build a fully functional 3-role bus reservation system with a professional GUI
- Implement and demonstrate 6 core DSA concepts: Hash Table, BST, Queue, Graph (Dijkstra + BFS), Merge Sort, and Bubble Sort
- Enforce robust input validation across every user-facing form in the system
- Integrate SQL Server for persistent, relational data storage across sessions
- Generate professional PDF tickets and invoices on booking confirmation
- Implement Role-Based Access Control (RBAC) for Admin, Manager, and Passenger roles

1.2 Tools and Technologies

Technology	Purpose
Python 3.x	Primary programming language
tkinter	GUI framework for all windows, forms, treeviews and panels
SQL Server Express	Relational database for persistent storage
pyodbc	Python-to-SQL Server database connector
ReportLab	PDF generation library for tickets and invoices
heapq (Python stdlib)	Priority queue used in Dijkstra's algorithm
csv (Python stdlib)	Waitlist logging and bookings CSV export
GitHub	Version control and project tracking

2. Database Design

The application uses SQL Server Express as its relational database engine.

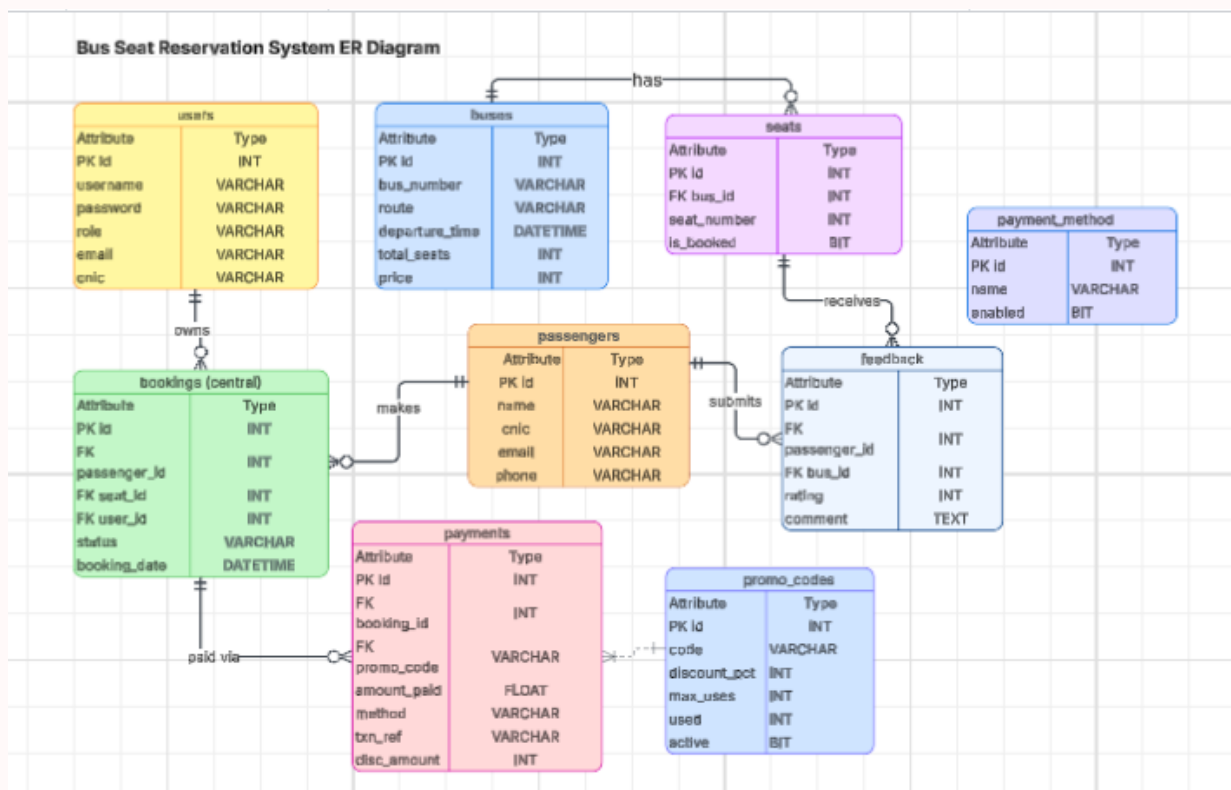
All database interactions are centralized in a single module called `database.py` — the Data Access Layer — ensuring a clean separation between the interface and data concerns. No SQL is written outside this file.

2.1 Entity Relationship Overview

The database is organized around the central bookings table, which connects passengers, seats, buses, and users. Payments are linked to bookings, feedback is linked to passengers and buses, and promo codes are referenced at the payment stage.

Key relationships:

- A Bus has many Seats (1:N)
- A Passenger can have many Bookings (1:N)
- A Booking belongs to one Seat and one Passenger (N:1)
- A Payment belongs to one Booking (1:1)
- A User account is linked to Bookings via `user_id` (1:N)
- Feedback is submitted by a Passenger for a Bus (N:M resolved via feedback table)
- Payments optionally reference a Promo Code (many Payments to one Promo Code)



2.2 Table Descriptions

The database contains 9 tables. The table below summarises each one:

Tables	Key Columns	Purpose
buses	id, bus_number, route, departure_time, total_seats, price	Stores all bus records with route and schedule information
seats	id, bus_id FK, seat_number, is_booked	Individual seat records linked to each bus
passengers	id, name, cnic, email, phone	Passenger personal information collected at booking
bookings	id, passenger_id FK, seat_id FK, user_id FK, status, booking_date	Core booking record linking passenger, seat and user account
users	id, username, password, role, email, cnic	Login accounts for all three roles
payments	id, booking_id FK, promo_code FK, amount_paid, method, txn_ref	Payment records per booking with promo reference
feedback	id, passenger_id FK, bus_id FK, rating, comment	Star ratings and comments submitted per bus
promo_codes	id, code, discount_pct, max_uses, used, active	Discount codes created and managed by Admin
payment_methods	id, name, enabled	Toggable payment options managed by Admin

3. Architecture Overview

BusRes is architected across six distinct layers, each with a clear and non-overlapping responsibility. This separation ensures the codebase remains maintainable, testable, and extensible.

3.1. Presentation Layer:

The Presentation Layer is everything the user sees and interacts with. It is built entirely using Python's tkinter library and consists of three window classes and all panel methods inside BusResApp.

- LoginWindow — application entry point with role selection, sign-in form, and passenger registration
- BusResApp — main application window containing all 40+ panels as individual methods
- CreateAccountWindow, ChangePasswordWindow — modal popup windows for account management
- All forms, treeviews, buttons, the Coffee & Caramel colour theme, and layout management live here

Responsibility: Display data, collect user input, and surface validation errors inline.



3.2. Business Logic Layer:

The Business Logic Layer encodes all application rules and decision-making. It lives inside the methods of BusResApp in main.py alongside the presentation code.

- “A passenger can only book using their registered CNIC”
- “If a bus is full, the booking automatically joins the waitlist”
- “A cancelled seat cannot be re-confirmed if another passenger now occupies it”
- “Admin must enter a password to access the waitlist panel”
- “Manager can edit buses but cannot access the Admin Panel”
- “Promo discounts are calculated and applied before payment is processed”

Responsibility: Enforce rules, validate inputs, coordinate between layers, and make decisions.

3.3. Data Access Layer:

All database communication is handled exclusively by database.py. No SQL is written anywhere else in the application, ensuring a single point of database responsibility.

- Centralises all 50+ database functions: get_all_bookings(), book_seat(), add_passenger() and so on
- Manages all database connections via the get_connection() function
- Hides SQL complexity from the rest of the application entirely
- Handles cascading deletes and referential integrity in Python code before issuing SQL

Responsibility: Read from and write to the database, and abstract SQL complexity away from other layers.

3.4. Data Layer:

The Data Layer is the SQL Server Express database itself — the permanent, persistent store for all application data.

- 9 tables with foreign key relationships enforced at the database level
- Cascading deletes ensure referential integrity on bus deletion
- User passwords are stored and validated with case-sensitivity enforced at the Python level

Responsibility: Persist all data permanently across application sessions.



3.5. DSA Layer----The Core Intelligence Engine:

The DSA Layer is unique to this project and represents its academic core. All six data structures operate entirely in RAM (in-memory), are loaded on startup from the database, and are updated in real-time alongside the database on every booking action. This hybrid approach means the database guarantees data is never lost, while the DSA structures guarantee the application never feels slow.

DSA Module	File	Structure	Role in System
BookingHashTable	dsa/searching.py	Hash Table	O(1) booking lookup by ID in Search panel
PassengerBST	dsa/bst.py	Binary Search Tree	O(log n) passenger name search
WaitlistQueue	dsa/queue_waitlist.py	FIFO Queue	Manages waitlisted passengers in arrival order
RouteGraph	dsa/graph.py	Weighted Directed Graph	Dijkstra shortest path + BFS traversal
SeatMatrix	dsa/seat_matrix.py	2D Matrix	Seat layout representation per bus
compare_sorts()	dsa/sorting.py	Merge Sort + Bubble Sort	Algorithm comparison with live timing stats

3.6. File & Export Layer:

The File and Export Layer handles all file system operations outside the database.

- file_handler.py — exports all bookings to a CSV file on demand
- _generate_ticket_pdf() — generates a dark-themed PDF ticket using ReportLab on booking confirmation
- _generate_invoice_pdf() — generates a matching PDF invoice
- waitlist_log.csv — logs every waitlist action (add / serve) with timestamp

All generated files are saved to D:\BusSeatReservation\ and auto-opened after generation.

Responsibility: Generate downloadable artifacts and maintain persistent file-based logs.

4. Main Code Discussion

4.1 Python Modules:

In Python, every .py file is a module — a self-contained unit of code with a specific responsibility. BusRes is organized into the following modules:

Module	Layer	Responsibility
main.py	Presentation + Business Logic	All UI classes, panel methods, business rules, PDF generation
database.py	Data Access Layer	All 50+ database functions, SQL queries, connection management
file_handler.py	File & Export Layer	CSV export of all bookings
dsa/searching.py	DSA Layer	BookingHashTable — $O(1)$ hash-based booking search
dsa/bst.py	DSA Layer	PassengerBST — binary search tree by passenger name
dsa/queue_waitlist.py	DSA Layer	WaitlistQueue — FIFO queue for waitlisted passengers
dsa/graph.py	DSA Layer	RouteGraph — Dijkstra and BFS on city graph
dsa/seat_matrix.py	DSA Layer	SeatMatrix — 2D matrix seat layout per bus
dsa/sorting.py	DSA Layer	compare_sorts() — Merge Sort vs Bubble Sort benchmarking

The dsa/ folder groups all DSA modules together as a Python package, imported by main.py at startup. The separation means no DSA logic leaks into the UI code and no UI logic leaks into the data structures.

4.2 Class Structure in main.py:

main.py contains exactly four Python classes — three UI classes and one core application class. The three UI classes handle specific windows, while BusResApp is the entire application:

```
# Three UI window classes:
class LoginWindow(tk.Tk)
class CreateAccountWindow
class ChangePasswordWindow

# Core application class:
class BusResApp(tk.Tk)
methods

# Entry point window — login + role selection
# Modal popup — passenger registration
# Modal popup — password update

# Main app — contains all 40+ panels as
```

BusResApp inherits from tk.Tk, making it the root application window. Every sidebar button the user sees corresponds to exactly one method inside BusResApp. Navigation works by calling `_clear()` to destroy all current widgets, then calling the target method to rebuild the panel fresh with updated data:

```
def _clear(self):
    for widget in self.main.wininfo_children():
        widget.destroy()          # wipe current panel

# Then the new panel method builds everything from scratch:
def _show_analytics(self):        # Admin: live analytics dashboard
    self._clear()
    self._heading('Analytics Dashboard', ...)
    # fetch fresh data, build widgets...
```

4.3 Key Panel Methods:

The following table lists the most important panel methods inside BusResApp by role:

Method	Role	What it renders
<code>_show_analytics()</code>	Admin	Animated stat cards + bar charts for bookings per bus and per route
<code>_show_revenue()</code>	Admin	Revenue totals, per-bus revenue list and bar chart sorted by bus number
<code>_show_promo_codes()</code>	Admin	Promo code table with create/toggle/delete actions
<code>_show_user_mgmt()</code>	Admin	All user accounts with role-colour coding
<code>_show_all_buses_admin()</code>	Admin	All buses with live search bar and full/available status
<code>_show_admin_panel()</code>	Admin	All bookings with search, cancel/confirm, and waitlist popup
<code>_show_manage_buses()</code>	Manager	Bus list with Edit and Delete, red rows for full buses
<code>_show_add_bus()</code>	Manager	Form to add a new bus with format hints on every field
<code>_show_feedback_monitor()</code>	Manager	Average star ratings per bus + full feedback table
<code>_show_all_buses_passenger()</code>	Passenger	All buses with route filter, red rows for full buses
<code>_show_book_seat()</code>	Passenger	Two-column PanedWindow: bus list left + booking form right
<code>_show_my_bookings()</code>	Passenger	Personal booking history by user_id with total spent
<code>_show_my_waitlist()</code>	Passenger	Personal waitlist entries with queue position
<code>_show_search()</code>	All roles	Hash Search, Binary Search and BST Search in one panel
<code>_show_routes()</code>	All roles	Dijkstra route finder + BFS traversal + distance table
<code>_show_sort_stats()</code>	All roles	Merge Sort vs Bubble Sort live benchmarking on real data

4.4 Startup sequence

Every time the application launches, four self-healing functions execute before any UI is displayed. This makes the application resilient to schema changes and data inconsistencies without requiring manual database intervention:

```
ensure_schema()           # Creates any missing tables if not present
add_user_id_to_bookings() # Adds user_id column to bookings if missing
sync_seats_from_bookings() # Corrects seat is_booked flags from booking
                           records
backfill_payments()       # Generates payment records for bookings
                           missing one
```

4.5 Role-Based Access Control (RBAC):

Three completely different dashboards are served from a single login screen. The sidebar navigation list, available panels, and data access scope all change dynamically at runtime based on the authenticated role:

```
if self.current_role == 'admin':
    nav = [Analytics, Revenue, Promo Codes, User Mgmt, All Buses, ...]
elif self.current_role == 'manager':
    nav = [Manage Buses, Add Bus, Feedback Monitor, Search, ...]
else:   # passenger
    nav = [All Buses, Book a Seat, My Bookings, My Waitlist, Feedback, ...]
```

4.6 Event-Driven Architecture:

The entire UI is event-driven — nothing happens unless triggered by a user action. Three types of event bindings are used throughout the application:

```
command = self._show_analytics           # Button click fires a method
q_var.trace_add('write', _filter)         # Entry change triggers live
                                         search
tree.bind('<<TreeviewSelect>>', f)         # Row selection fires a callback
```

This pattern means the interface is always responsive. Live search results in the Admin Panel and the booking suggestion dropdown in Hash Search both update on every single keystroke without any explicit button press.

4.7 Key Method Call chains:

The following system chains discussed here illustrate how a single user action triggers a coordinated sequence across multiple layers of the application:



Booking a Seat

```
Click 'Book / Join Waitlist'
→ _book() # validate all fields
→ _show_payment_popup() # display order summary + payment
                        options
→ _pay() # process payment with 1.2s animation
→ add_passenger() # DAL: create or find existing
                        passenger
→ book_seat() /add_waitlist_booking() # DAL: create booking record
→ record_payment() # DAL: save payment with promo if
                        applied
→ hash_table.insert() # DSA: update hash table in memory
→ passenger_bst.insert() # DSA: update BST in memory
→ waitlist.enqueue() # DSA: add to queue if bus is full
→ _show_success() # show confirmation popup
→ _generate_ticket_pdf() # File layer: create and open PDF
                        ticket
```

Delete Bus

```
Click 'Delete'
→ _delete_bus_action()
→ DELETE payments (cascade step 1 – must go first)
→ DELETE bookings (cascade step 2)
→ DELETE seats (cascade step 3)
→ DELETE bus (cascade step 4)
→ _show_manage_buses() # refresh panel
```

Find Shortest Route

```
Click 'Find Shortest Route'
→ route_graph.dijkstra(from, to) # DSA: priority queue pathfinding
→ _get_edge_weight(u, v) # fetch km for each path segment
→ route_graph.bfs(from) # DSA: BFS level-order traversal
→ render distance table # all cities with distances from source
→ render BFS card # traversal order + plain-English
explanation
```

Serving Waitlist

```
Click 'Serve Next'
→ waitlist.peek() # DSA: inspect front of queue
→ serve_waitlist_booking() # DAL: assign the freed seat
→ waitlist.dequeue() # DSA: remove from front of queue
→ hash_table.insert() # DSA: update record in hash table
→ passenger_bst.insert() # DSA: update record in BST
→ _show_admin_panel() # refresh admin view instantly
```

4.8 In-Memory and Database Hybrid

DSA structures (Hash Table, BST, Queue) live in RAM for speed. The database stores data permanently. On startup, the RAM structures are populated from the database. Every new booking updates both simultaneously:

- Database — permanent persistence; data survives app restart
- Hash Table / BST — instant in-memory search without SQL queries on every keystroke
- Queue — ordered FIFO waitlist management in real time

The RAM structures act as a high-speed cache layer over the database, enabling $O(1)$ and $O(\log n)$ lookups that would otherwise require full table scans on every user interaction. When the app closes, RAM clears. On the next startup it reloads from the database fresh — same data, rebuilt instantly.

4.9 Input Validation:

Input validation is implemented on every user-facing form using a combination of Python regex pattern matching and logical business rule checks. All validation errors display inline next to the relevant field — no disruptive popup dialogs for field errors.

Panel	What is Validated
Login	Username and password required; role must match the account type exactly
Create Account	Username required; password 4-20 characters; confirmation passwords must match
Book a Seat	Name must match account username; CNIC format 12345-1234567-1; valid email; phone format 0301-1234567; CNIC must be the registered CNIC for that account
Waitlist (Admin)	Same CNIC, phone, and name validation rules as the booking form
Add Bus	All fields required; total seats ≥ 1 ; price ≥ 1
Edit Bus	Same as Add Bus; price cleaned of PKR/comma formatting before digit validation
New Promo Code	Discount value 1-100%; max uses ≥ 1
Change Password	4-20 characters; confirmation must match new password
Feedback	Passenger must have at least one confirmed booking to submit feedback

Regex patterns used:

```
r'\d{5}-\d{7}-\d{'          # CNIC:    12345-1234567-1
r'0\d{3}-\d{7}'           # Phone:    0301-1234567
r'[^@]+@[^@]+\.[^@]+'      # Email:    user@domain.com
```

These patterns use Python's `re.fullmatch()` for strict end-to-end matching — partial matches are rejected outright.

5. Core DSA Implementations & their Role:

This section describes each of the six data structures and algorithms implemented in the project, their purpose, time complexity, and exactly how they operate within the BusRes system.

5.1. Hash Table — $O(1)$ Booking Search

The `BookingHashTable` stores all booking records with the Booking ID as the key and the full booking record as the value. When a user types a Booking ID in the Search panel, the hash table returns the result in constant time $O(1)$ — regardless of how many thousands of bookings exist in the database.

A live dropdown suggestion list updates with every single keystroke, powered entirely by the in-memory hash table without any database query. This is only possible because of the $O(1)$ access time.

- Class: `BookingHashTable` in `dsa/searching.py`
- Key method: `hash_table.search(booking_id)`
- Loaded on startup: `_load_hash_table()` populates from all DB bookings
- Updated on every new booking: `hash_table.insert(bid, booking_record)`

Code Snippet:

```
def _load_hash_table(self):
    for b in get_all_bookings():
        self.hash_table.insert(b["id"], b)
```

5.2. Binary Search Tree (BST) — $O(\log n)$ Passenger Search

The `PassengerBST` stores passenger names as keys with their booking records as values. Every time a booking is made, the passenger's name is inserted into the tree. The left subtree always contains alphabetically smaller names and the right subtree alphabetically larger names.

Search example — looking for 'Ali' in a tree rooted at 'Maria':

1. Start at root 'Maria' — `Ali < Maria` → go left
2. Find 'Ali' → return their booking record instantly

If a passenger name is not found in the BST — for example because it was booked in a previous app session before the tree was rebuilt — the system falls back to a full database search as a safety net. The BST is always the fast path; the database is the reliable backup.

- Class: `PassengerBST` in `dsa/bst.py`
- Time complexity: $O(\log n)$ average case
- Key methods: `passenger_bst.search(name)`, `passenger_bst.insert(name, record)`

Code Snippet:

```
# Load BST on startup
def _load_bst(self):
    for b in get_all_bookings():
        self.passenger_bst.insert(b["name"], b)

# Search BST
r = self.passenger_bst.search(name)
```

5.3. Binary Search on Seat Numbers

The Binary Search feature accepts a seat number, sorts all bookings by seat number first, then performs classic binary search with low/high/mid pointers.

The most recent booking for that seat number is highlighted with a star symbol.

- All confirmed bookings sorted by seat_number before search: $O(n \log n)$
- Binary search: if mid value < target $\rightarrow$ low = mid+1, if > target $\rightarrow$ high = mid-1
- Overall time complexity: $O(n \log n)$ for sort + $O(\log n)$ for search

Code Snippet:

```
bookings = sorted(all_b, key=lambda x: int(x["seat_number"]))
low, high, hit_idx = 0, len(bookings)-1, -1
while low <= high:
    mid = (low + high) // 2
    mid_val = int(bookings[mid]["seat_number"])
    if mid_val == target: hit_idx = mid; break
    elif mid_val < target: low = mid + 1
    else: high = mid -
```

5.4. Waitlist Queue — FIFO Management

The WaitlistQueue implements a classic First In First Out queue for managing waitlisted passengers. When a bus is full and a passenger books, they are enqueued at the back of the queue. When an Admin clicks Serve Next, the passenger at the front of the queue is dequeued and automatically assigned the freed seat.

- Class: WaitlistQueue in dsa/queue_waitlist.py
- Operations: enqueue() adds to back, dequeue() removes from front, peek() inspects front without removing
- Passenger A who waitlists before Passenger B is always served first — guaranteed by FIFO order
- The queue syncs with the database on every enqueue and dequeue

Code Snippet:

```
passenger = self.waitlist.peek()      # inspect front
self.waitlist.dequeue()               # remove from front
self.waitlist.enqueue({...})         # add to back# Search BST
```

4.1 Sorting Algorithm Comparison — Merge Sort vs Bubble Sort

The Sort & Stats panel benchmarks Merge Sort against Bubble Sort on the real live booking dataset. Each algorithm is run five times and the average execution time in milliseconds is reported, along with exact comparison counts, swap counts, and a speed ratio.

Algorithm	Time Complexity	What is Counted	Typical Result
Merge Sort	$O(n \log n)$	Comparisons during merge + divide steps	Significantly faster on all dataset sizes
Bubble Sort	$O(n^2)$	Adjacent comparisons + swap operations	Much slower; performance degrades rapidly with data size

Both algorithms are instrumented — comparison and swap counters are incremented inside the sort logic itself during every operation, giving precise real counts rather than theoretical estimates. The verdict displayed reads: “Merge Sort is the industry standard. Bubble Sort is $O(n^2)$ — doubling data quadruples the work.”

Code Snippet for Merge Sort:

```
def _merge_sort_inst(data):
    if len(data) <= 1: return data
    mid = len(data) // 2
    merge_merges[0] += 1
    return _merge_inst(
        _merge_sort_inst(data[:mid]),
        _merge_sort_inst(data[mid:]))
```

Code Snippet for Bubble Sort:

```
def _bubble_sort_inst(data):
    data = data.copy()
    for i in range(len(data)):
        for j in range(0, len(data)-i-1):
            bubble_comparisons[0] += 1
            if str(data[j][key]) > str(data[j+1][key]):
                data[j], data[j+1] = data[j+1], data[j]
                bubble_swaps[0] += 1
    return data
```

4.2 Graph Algorithms— Dijkstra and BFS

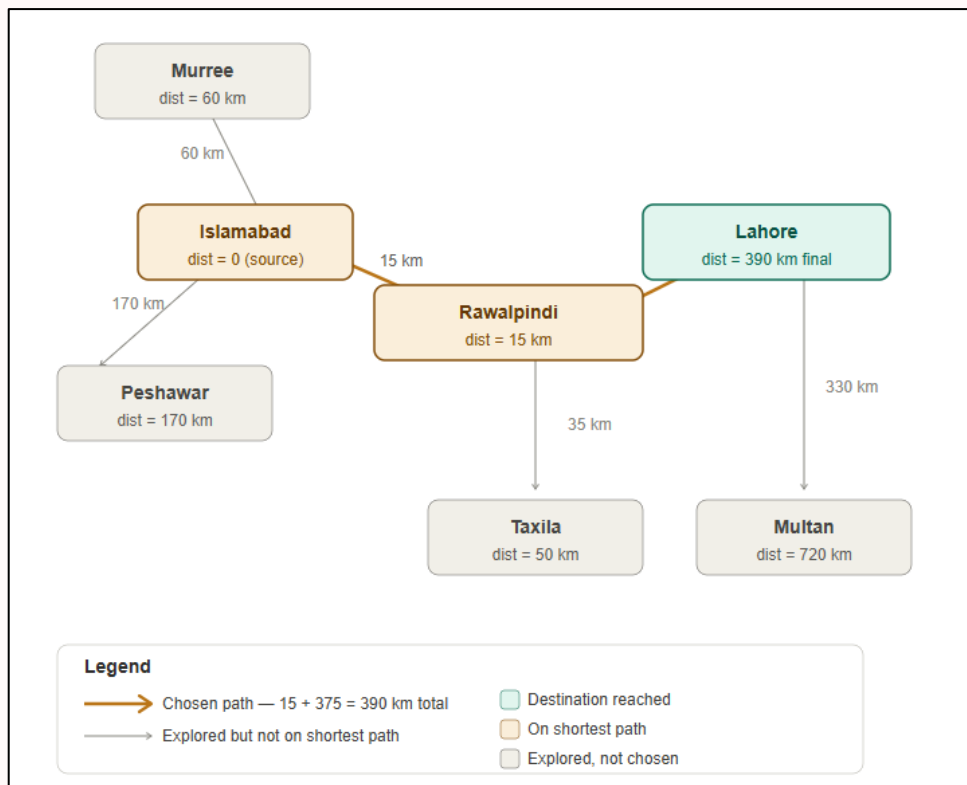
The Route Finder panel implements two graph algorithms on a weighted directed graph of 19 Pakistani cities connected by road distances in kilometres.

The graph is stored as an adjacency list:

```
self.graph = {
    'Islamabad': [('Rawalpindi', 15), ('Murree', 60), ('Peshawar',
170)],
    'Rawalpindi': [('Lahore', 375), ('Taxila', 35), ('Attock', 90)],
    # ... 19 city connections total
}
```

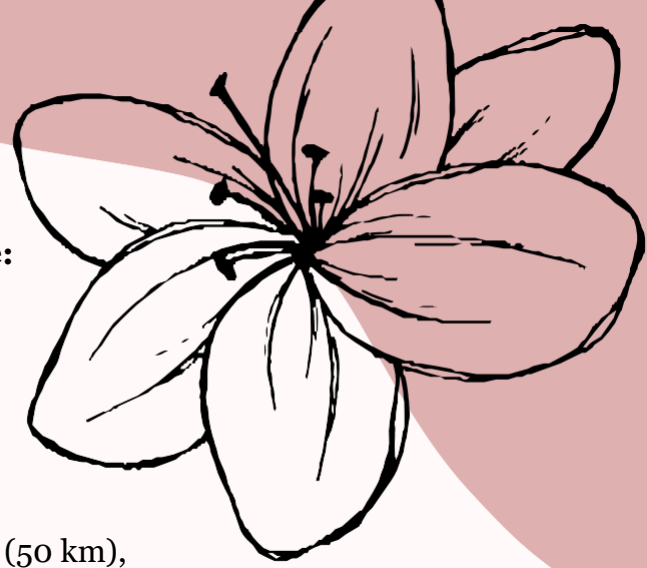
Dijkstra's Algorithm:

Dijkstra's algorithm finds the shortest path between two cities by total distance in kilometres. A min-heap priority queue (Python's `heapq` module) ensures the algorithm always processes the globally cheapest unvisited city next, guaranteeing the optimal result.



Step-by-step example — Islamabad to Lahore:

1. Start at Islamabad, distance = 0
2. Check all neighbours: Rawalpindi (15 km), Murree (60 km), Peshawar (170 km)
3. Pick cheapest unvisited: Rawalpindi at 15 km
4. From Rawalpindi: Lahore (15+375=390 km), Taxila (15+35=50 km)
5. Continue exploring all cheaper cities first: Taxila (50 km), Murree (60 km) ...
6. Lahore is eventually finalized at 390 km as the confirmed shortest path



The full Dijkstra distance table is rendered on screen showing every city's shortest distance from the source, with shortest-path cities highlighted in green. The algorithm explores every cheaper city before confirming the destination — this exhaustive exploration is precisely what guarantees optimality.

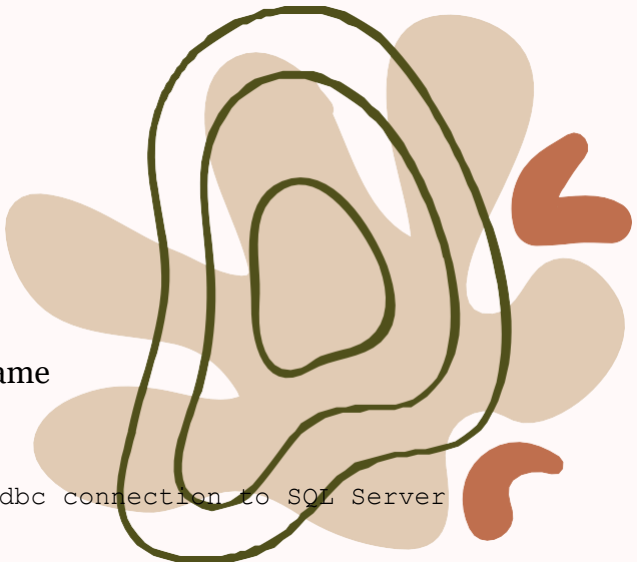
Breadth-First Search (BFS)

BFS is run from the source city and displayed below the Dijkstra results as a traversal card. BFS explores all directly connected cities first before moving to the next level — like ripples spreading outward from a stone in water. It guarantees visiting all reachable cities but does not guarantee the shortest distance by kilometres. BFS minimises hops; Dijkstra minimises total distance. The panel includes a plain-English explanation card explaining this distinction to the user.

6. Data Integration — Frontend and Backend

The application achieves seamless integration between the tkinter frontend and the SQL Server backend through the Data Access Layer (database.py) and the in-memory DSA layer working in tandem.





6.1 Connection Architecture

Every database operation in database.py follows the same consistent pattern:

```
def function_name(params):  
    conn = get_connection()      # Opens pyodbc connection to SQL Server  
    cursor = conn.cursor()  
    cursor.execute('SQL...', params)  
    result = cursor.fetchone()   # or fetchall() for multiple rows  
    conn.commit()                # For write operations only  
    conn.close()                 # Always closed immediately after use  
    return result
```

The connection string uses Windows Authentication (Trusted_Connection=yes) to connect to the local SQL Server Express instance. No credentials are hardcoded in the connection string, keeping it secure by default.

6.2 Dual-Write Strategy

Every write operation updates both the database and the in-memory DSA structures. This ensures the search panels always reflect the latest state without requiring a page reload or re-query:

```
# Step 1: Write to database (persistent)  
bid = book_seat(passenger_id, seat_id, user_id)  
  
# Step 2: Update in-memory structures simultaneously (fast access)  
self.hash_table.insert(bid, booking_record)  
self.passenger_bst.insert(passenger_name, booking_record)
```

This dual-write pattern guarantees consistency between the permanent database state and the in-memory search structures at all times during an application session.

7. UI Design Overview

The application uses a custom Coffee & Caramel dark theme designed to provide a warm, professional aesthetic that is easy on the eyes during extended use. The theme is defined as a set of global constants at the top of main.py and applied consistently across every widget.



7.1 Colour Palette

Constant	Hex Value	Used For
BG	#1CoFoA	Main application background — very dark brown
PANEL	#2C1810	Cards, treeview background, form panels
SIDEBAR	#241209	Navigation sidebar background
ACCENT	#6B3A2A	Buttons, borders, selected states
GOLD	#E8A44A	Highlights, selected treeview rows
GREEN	#7D9B6A	Success states, confirm buttons, available indicators
RED	#C0392B	Error states, cancel buttons, full bus row text
YELLOW	#C8853A	Subtitles, labels, promo indicators
BLUE	#4A7C9E	Passenger role colour, PDF ticket button
PURPLE	#8B6F7A	Waitlisted status colour in treeview
BG	#1CoFoA	Main application background — very dark brown

7.2 Screen Descriptions

The following panels cover the key screens across all three roles:

Login Screen + Create Account Pop-up

BusRes — Sign In



BusRes

Sign in to your account

Select Role

☒ ADMIN
 ☐ MANAGER
 ☐ PASSENGER

Username

Maheen

Password

•••••

Sign In

New Passenger? Create an account below:

+ Create Account

Create Account



Create Account

Passenger registration only

Username

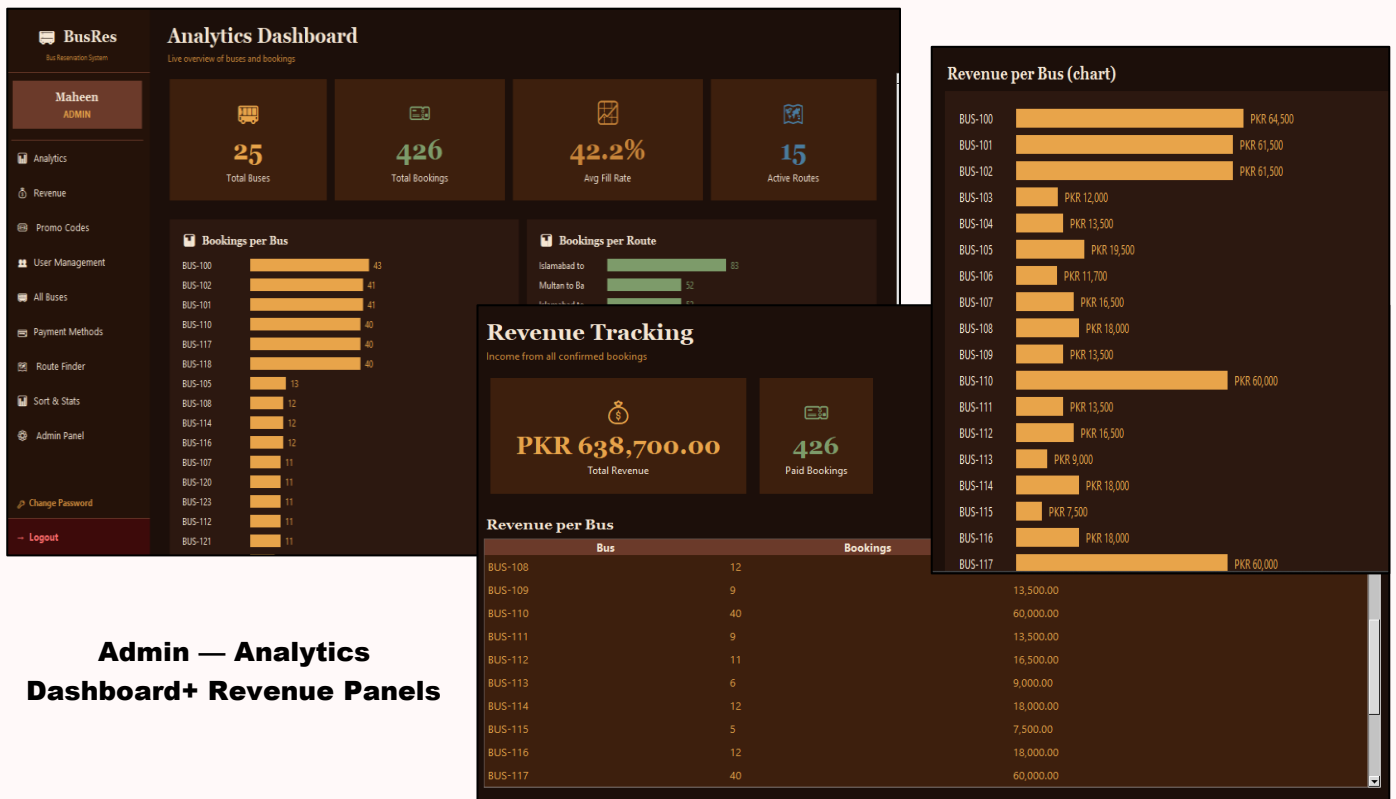
Email (optional)

Password (4-20 chars)

Confirm Password

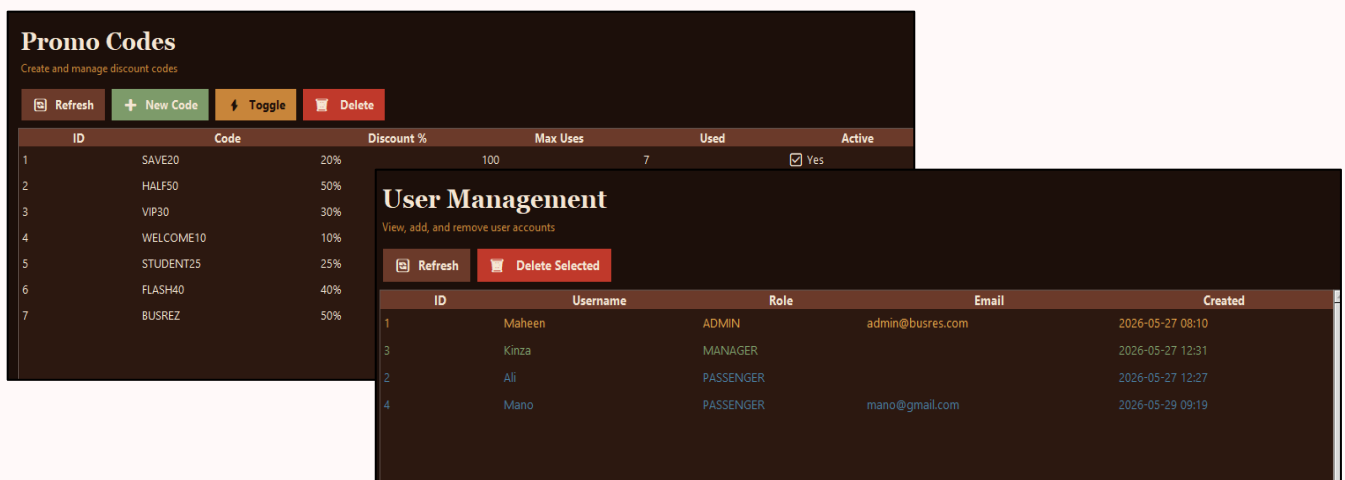
☒ Create Account

— Back to Login

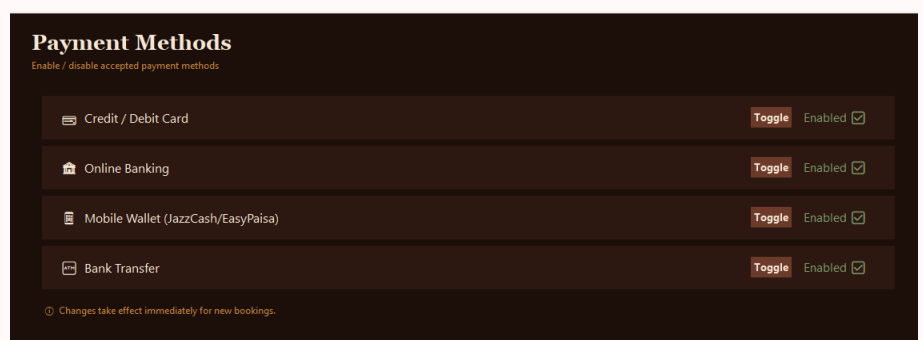


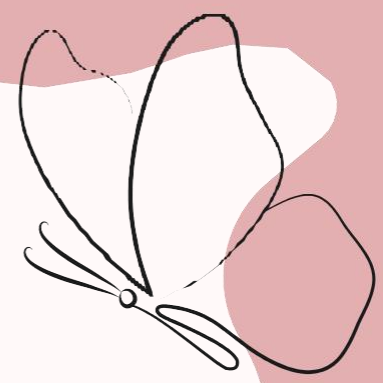
Admin — Analytics Dashboard+ Revenue Panels

Admin — Promo codes + User Management:



Admin — Payment Methods:





Admin /Manager —Sorts & Stats Panel+ All Buses Panel

Sorting Algorithm Comparison

Summary

Total records sorted: 428

Sorted by field: Booking ID (ascending)

Faster algorithm: Merge Sort

Merge Sort — $O(n \log n)$

Time taken (avg 5 runs): 4.8788 ms

Total comparisons: 12,426

Divide steps: 2,562

→ How it works: splits the list into halves recursively until each piece has 1 element, then merges pairs of records it split 2562 times and compared 12,426 pairs during merging.

IDs before (first 5): 1 2 3 4 5

IDs after (first 5): 1 10 100 101 102

Bubble Sort — $O(n^2)$

Time taken (avg 5 runs): 89.9453 ms

Total comparisons: 548,268

Total swaps: 159,828

→ How it works: repeatedly walks the list comparing adjacent pairs and swapping them if out of order. It takes 548,268 comparisons and 159,828 swaps — every out-of-order pair costs a swap, which is why it's slow on large lists.

All Buses

Read-only overview of all buses

ID	Bus No	Route	Departure	Seats	Booked	Available	Price	Status
1	BUS-100	Islamabad to Rawalpindi	2026-06-03 02:37	42	42	0	PKR 2,000	FULL
2	BUS-101	Rawalpindi to Lahore	2026-06-10 23:27	41	41	0	PKR 1,500	FULL
3	BUS-102	Islamabad to Murree	2026-06-17 06:31	41	41	0	PKR 1,500	FULL
4	BUS-103	Peshawar to Islamabad	2026-06-13 19:03	40	8	32	PKR 1,500	Available
5	BUS-104	Lahore to Multan	2026-06-08 15:13	40	9	31	PKR 1,500	Available
6	BUS-105	Karachi to Hyderabad	2026-06-22 11:11	40	13	27	PKR 1,500	Available
7	BUS-106	Quetta to Karachi	2026-05-29 04:34	40	8	32	PKR 1,500	Available
8	BUS-107	Faisalabad to Lahore	2026-06-05 18:35	40	11	29	PKR 1,500	Available
9	BUS-108	Multan to Bahawalpur	2026-05-28 08:04	40	12	28	PKR 1,500	Available
10	BUS-109	Abbottabad to Islamabad	2026-06-15 23:17	40	9	31	PKR 1,500	Available
11	BUS-110	Islamabad to Rawalpindi	2026-06-04 07:15	40	40	0	PKR 1,500	FULL
12	BUS-111	Rawalpindi to Lahore	2026-06-11 14:30	40	9	31	PKR 1,500	Available
13	BUS-112	Islamabad to Murree	2026-06-18 09:00	40	11	29	PKR 1,500	Available
14	BUS-113	Peshawar to Islamabad	2026-06-14 06:45	40	6	34	PKR 1,500	Available
15	BUS-114	Lahore to Multan	2026-06-09 11:20	40	12	28	PKR 1,500	Available
16	BUS-115	Karachi to Hyderabad	2026-06-23 16:00	40	5	35	PKR 1,500	Available
17	BUS-116	Quetta to Karachi	2026-05-30 21:10	40	12	28	PKR 1,500	Available
18	BUS-117	Faisalabad to Lahore	2026-06-06 08:50	40	40	0	PKR 1,500	FULL
19	BUS-118	Multan to Bahawalpur	2026-05-29 13:00	40	40	0	PKR 1,500	FULL

Admin /Manager —Route Finder Panel:

Route Finder — Dijkstra's Algorithm

From: Abbottabad

To: Attock

Find Shortest Route

Shortest Route Found

Route Segments:

Hop 1: Abbottabad → Murree 55 km

Hop 2: Murree → Rawalpindi 50 km

Hop 3: Rawalpindi → Taxila 35 km

Hop 4: Taxila → Attock 45 km

Total: 55 + 50 + 35 + 45 = 185 km

Dijkstra distance table:

City	Dist from Abbottabad	Via
★ Abbottabad	0 km	—
★ Murree	55 km	Abbottabad
★ Rawalpindi	105 km	Murree
Islamabad	115 km	Murree
★ Taxila	140 km	Rawalpindi
★ Attock	185 km	Taxila
Peshawar	280 km	Attock
Lahore	480 km	Rawalpindi
Faisalabad	610 km	Lahore

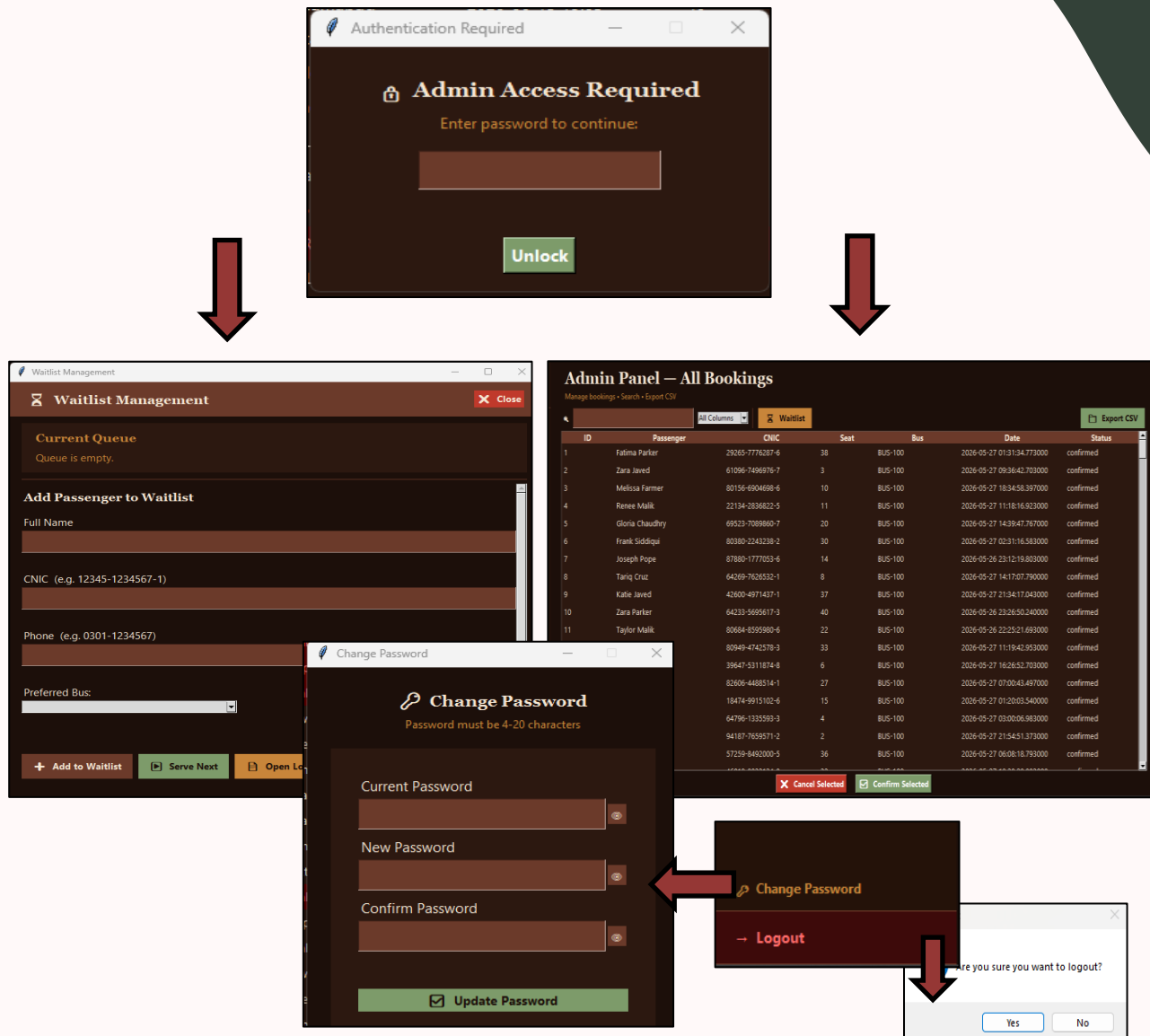
BFS Traversal from Abbottabad

Abbottabad → Islamabad → Murree → Rawalpindi → Peshawar → Taxila → Lahore → Attock → Multan → Faisalabad → Bahawalpur → Sukkur → Quetta → Karachi → Hyderabad

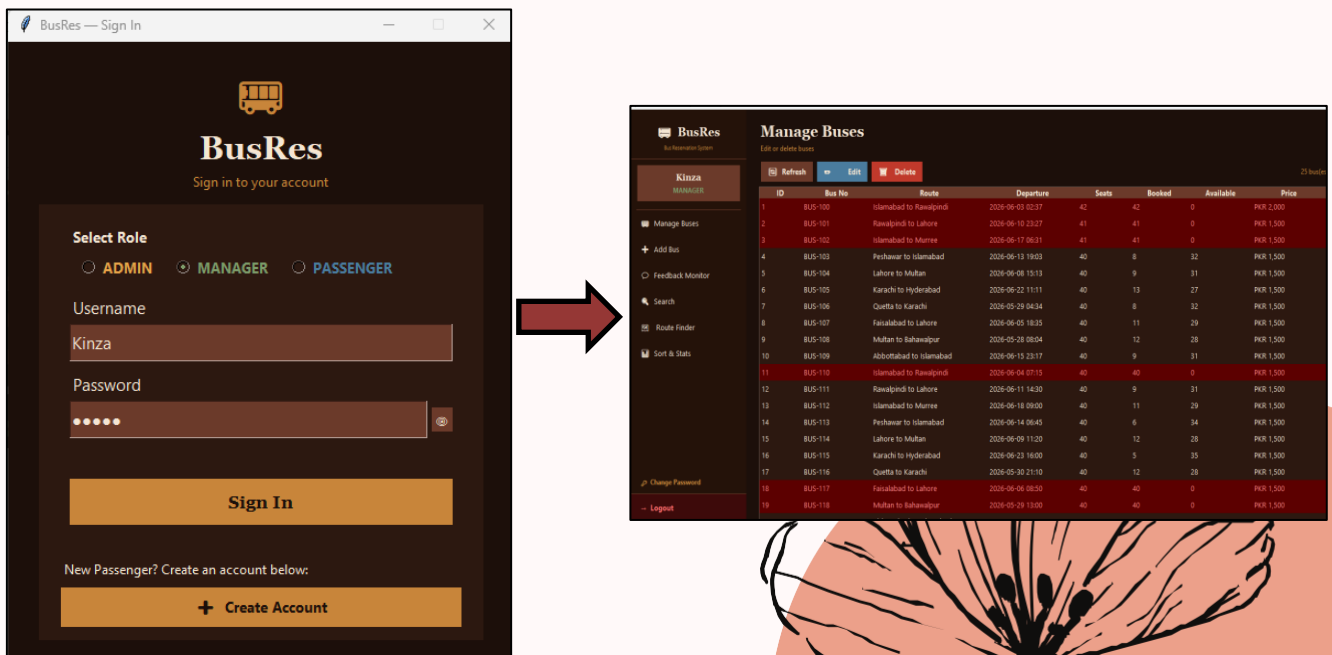
What is BFS?

BFS (Breadth-First Search) explores all directly connected cities first, then moves to the next level — like ripples spreading outward from a stone in water. It visits every reachable city but does NOT guarantee the shortest distance. That is Dijkstra's job. BFS only cares about HOW MANY hops, not how many kilometres.

Access Flow to Admin Panel + Admin Waitlist + Change Pass+ Logout Panels



Flow To Manager—Manage Buses Panel From Login Window:



Manager---Edit Buses Panel + Add Bus Panel:

Edit Bus

Editing BUS-109

Bus Number (e.g. BUS-125)

BUS-109

Route (e.g. Lahore to Karachi)

Abbottabad to Islamabad

Departure Time (e.g. 2026-07-01 08:30)

2026-06-15 23:17

Total Seats (e.g. 40)

40

Price (PKR)

1500

Save

Cancel

Add New Bus

Fill in the details below

Bus Number (e.g. BUS-125)

Route (e.g. Lahore to Karachi)

Departure Time (e.g. 2026-07-01 08:30)

Total Seats (e.g. 40)

Price (PKR)

1500

+ Add Bus

Clear

Manager---Manage Buses Panel + Add Bus Panel:

Feedback Monitoring

View all passenger feedback

Average Ratings per Bus

BUS-100 — Islamabad to Rawalpindi	★★★★★	5.0/5 (1 reviews)
BUS-103 — Peshawar to Islamabad	★★★★☆	2.0/5 (1 reviews)
BUS-104 — Lahore to Multan	☆☆☆☆☆	0/5 (0 reviews)
BUS-105 — Karachi to Hyderabad	☆☆☆☆☆	0/5 (0 reviews)
BUS-106 — Quetta to Karachi	☆☆☆☆☆	0/5 (0 reviews)
BUS-107 — Faisalabad to Lahore	☆☆☆☆☆	0/5 (0 reviews)
BUS-108 — Multan to Bahawalpur	☆☆☆☆☆	0/5 (0 reviews)
BUS-109 — Abbottabad to Islamabad	☆☆☆☆☆	0/5 (0 reviews)
BUS-110 — Islamabad to Rawalpindi	☆☆☆☆☆	0/5 (0 reviews)
BUS-111 — Rawalpindi to Lahore	☆☆☆☆☆	0/5 (0 reviews)
BUS-112 — Islamabad to Murree	☆☆☆☆☆	0/5 (0 reviews)

All Feedback

ID	Passenger	Bus	Route	Rating	Comment	Date
2	Ali	BUS-100	Islamabad to Rawalpindi	★★★★★	nice bus	2026-05-31 15:18
1	Ali	BUS-103	Peshawar to Islamabad	★★★★☆		2026-05-28 20:27

Manager/Passenger---Search Panel+ Change Pass Pop-Up:

Search Bookings

Hash Search — Type a Booking ID:

Type the ID — matching bookings appear below in O(1).

160

Hash Search

Binary Search — Enter Seat Number:

16

Binary Search

BST Search — Enter Passenger Name:

Ali

BST Search

☒ BST Found — Ali | Seat: 1 | Bus: BUS-109 | Status: confirmed

☒ Seat 16 found on 13 bus(es):

- Rabia Sheikh | Bus: BUS-100 | Status: confirmed
- Katie Donaldson | Bus: BUS-101 | Status: confirmed
- Sadia Aguirre | Bus: BUS-102 | Status: confirmed
- Ahmed Banks | Bus: BUS-103 | Status: confirmed
- ★ Sheryl Pope | Bus: BUS-105 | Status: confirmed — most recent
- Melissa Hussain | Bus: BUS-107 | Status: confirmed
- Sadia Klein | Bus: BUS-109 | Status: confirmed
- Waqar Chaudhry | Bus: BUS-110 | Status: confirmed
- Hina Butt | Bus: BUS-111 | Status: confirmed
- ...

Change Password

Change Password

Password must be 4-20 characters

Current Password

New Password

Confirm Password

☒ Update Password

BusRes — Sign In



BusRes

Sign in to your account

Select Role

☐ ADMIN ☐ MANAGER ☒ PAS

Username

Ali

Password

••••

Sign In

New Passenger? Create an account below:

+ Create Account

All Buses

Browse all available routes

Filter by Route: All

ID	Bus No	Route	Departure	Capacity	Booked	Price	Status
1	BUS-100	Islamabad to Rawalpindi	2026-06-03 02:37	42	42	PKR 2,000	FULL
2	BUS-101	Rawalpindi to Lahore	2026-06-10 23:27	41	41	PKR 1,500	FULL
3	BUS-102	Islamabad to Murree	2026-06-17 06:31	41	41	PKR 1,500	FULL
4	BUS-103	Peshawar to Islamabad	2026-06-13 19:03	40	8	PKR 1,500	Available
5	BUS-104	Lahore to Multan	2026-06-08 15:13	40	9	PKR 1,500	Available
6	BUS-105	Karachi to Hyderabad	2026-06-22 11:11	40	13	PKR 1,500	Available
7	BUS-106	Quetta to Karachi	2026-05-29 04:34	40	8	PKR 1,500	Available
8	BUS-107	Faisalabad to Lahore	2026-06-05 18:35	40	13	PKR 1,500	Available
9	BUS-108	Multan to Bahawalpur	2026-05-28 08:04	40	12	PKR 1,500	Available
10	BUS-109	Abbottabad to Islamabad	2026-06-15 23:17	40	9	PKR 1,500	Available
11	BUS-110	Islamabad to Rawalpindi	2026-06-04 07:15	40	40	PKR 1,500	FULL
12	BUS-111	Rawalpindi to Lahore	2026-06-11 14:30	40	9	PKR 1,500	Available
13	BUS-112	Islamabad to Murree	2026-06-18 09:00	40	11	PKR 1,500	Available
14	BUS-113	Peshawar to Islamabad	2026-06-14 06:45	40	7	PKR 1,500	Available
15	BUS-114	Lahore to Multan	2026-06-09 11:20	40	13	PKR 1,500	Available
16	BUS-115	Karachi to Hyderabad	2026-06-23 16:00	40	5	PKR 1,500	Available
17	BUS-116	Quetta to Karachi	2026-05-30 21:10	40	12	PKR 1,500	Available
18	BUS-117	Faisalabad to Lahore	2026-06-06 08:50	40	40	PKR 1,500	FULL
19	BUS-118	Multan to Bahawalpur	2026-05-29 13:00	40	40	PKR 1,500	FULL
20	BUS-119	Abbottabad to Islamabad	2026-06-16 17:45	40	9	PKR 1,500	Available

Flow From Login to Passenger--- My Bookings Panel

Passenger—book a Seat Panel + Feedback Panel

Book a Seat

Select a bus below — FULL buses shown in red join the waitlist

Available Buses

ID	Bus No	Route	Departure	Seats	Avail	Price	Status
1	BUS-100	Islamabad to Rawalpindi	2026-06-03 02:37	42	0	PKR 2,000	FULL
2	BUS-101	Rawalpindi to Lahore	2026-06-10 23:27	41	0	PKR 1,500	FULL
3	BUS-102	Islamabad to Murree	2026-06-17 06:31	41	0	PKR 1,500	FULL
4	BUS-103	Peshawar to Islamabad	2026-06-13 19:03	40	32	PKR 1,500	32 left
5	BUS-104	Lahore to Multan	2026-06-08 15:13	40	31	PKR 1,500	31 left
6	BUS-105	Karachi to Hyderabad	2026-06-22 11:11	40	27	PKR 1,500	27 left
7	BUS-106	Quetta to Karachi	2026-05-29 04:34	40	32	PKR 1,500	32 left
8	BUS-107	Faisalabad to Lahore	2026-06-05 18:35	40	27	PKR 1,500	27 left
9	BUS-108	Multan to Bahawalpur	2026-05-28 08:04	40	28	PKR 1,500	28 left
10	BUS-109	Abbottabad to Islamabad	2026-06-15 23:17	40	31	PKR 1,500	31 left
11	BUS-110	Islamabad to Rawalpindi	2026-06-04 07:15	40	0	PKR 1,500	FULL
12	BUS-111	Rawalpindi to Lahore	2026-06-11 14:30	40	31	PKR 1,500	31 left
13	BUS-112	Islamabad to Murree	2026-06-18 09:00	40	29	PKR 1,500	29 left
14	BUS-113	Peshawar to Islamabad	2026-06-14 06:45	40	33	PKR 1,500	33 left
15	BUS-114	Lahore to Multan	2026-06-09 11:20	40	27	PKR 1,500	27 left
16	BUS-115	Karachi to Hyderabad	2026-06-23 16:00	40	35	PKR 1,500	35 left
17	BUS-116	Quetta to Karachi	2026-05-30 21:10	40	28	PKR 1,500	28 left
18	BUS-117	Faisalabad to Lahore	2026-06-06 08:50	40	0	PKR 1,500	FULL
19	BUS-118	Multan to Bahawalpur	2026-05-29 13:00	40	0	PKR 1,500	FULL
20	BUS-119	Abbottabad to Islamabad	2026-06-16 17:45	40	31	PKR 1,500	31 left

— Select a bus from the list

Passenger Details

Full Name

CNIC (12345-1234567-1)

Email

Phone (0301-1234567)

Promo Code (optional)

Book / Join Waitlist

Refresh

★ ★ ★ ★ ★ ★ ★ ★

Comment:

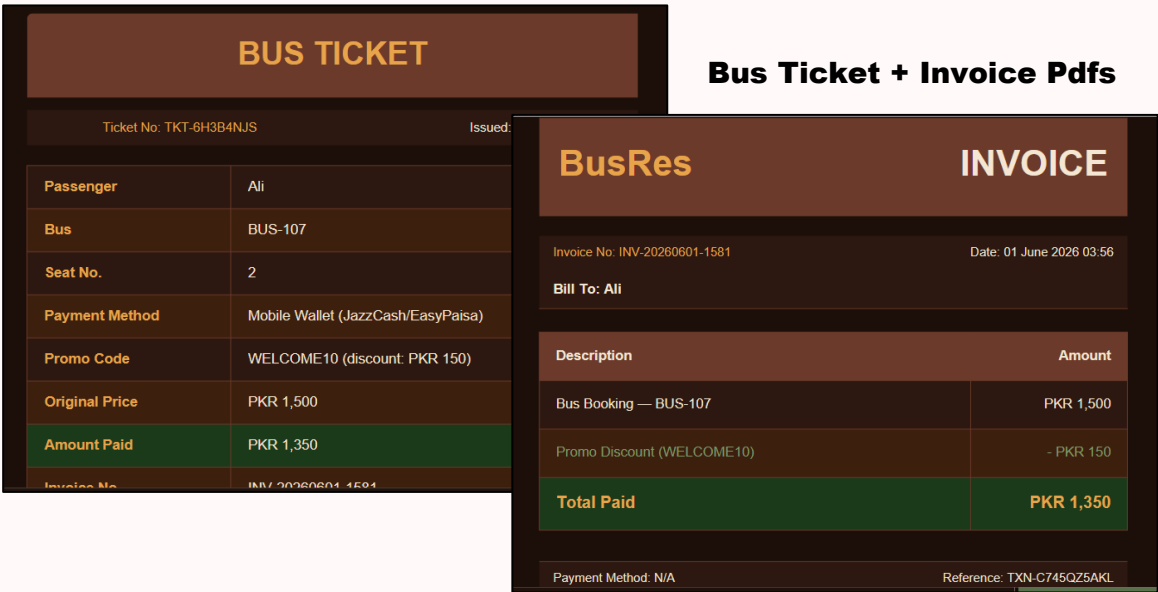
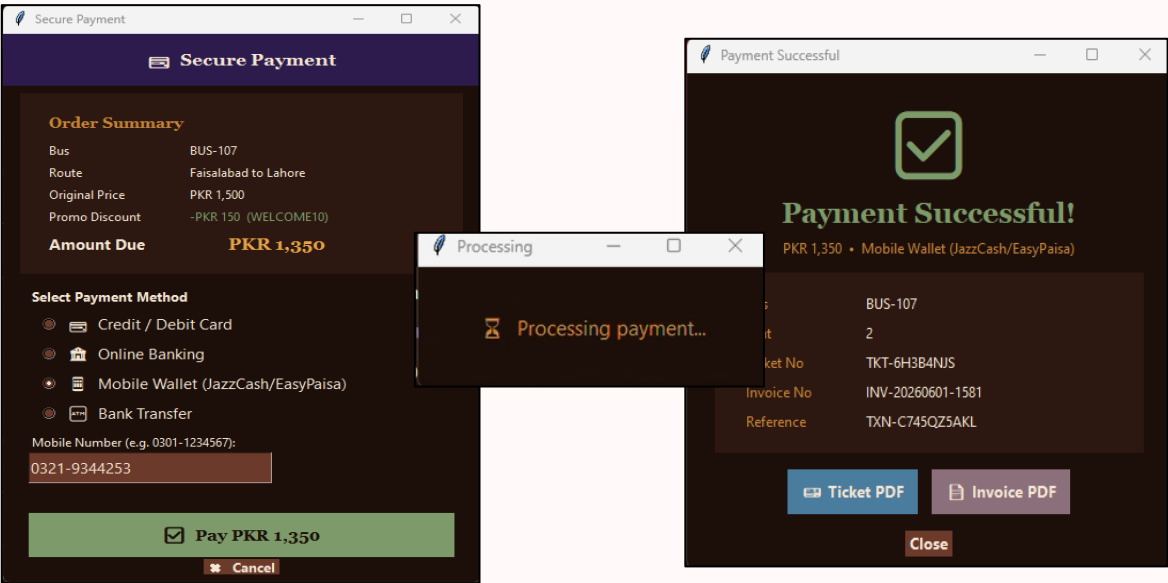
Submit Feedback

Your Previous Feedback

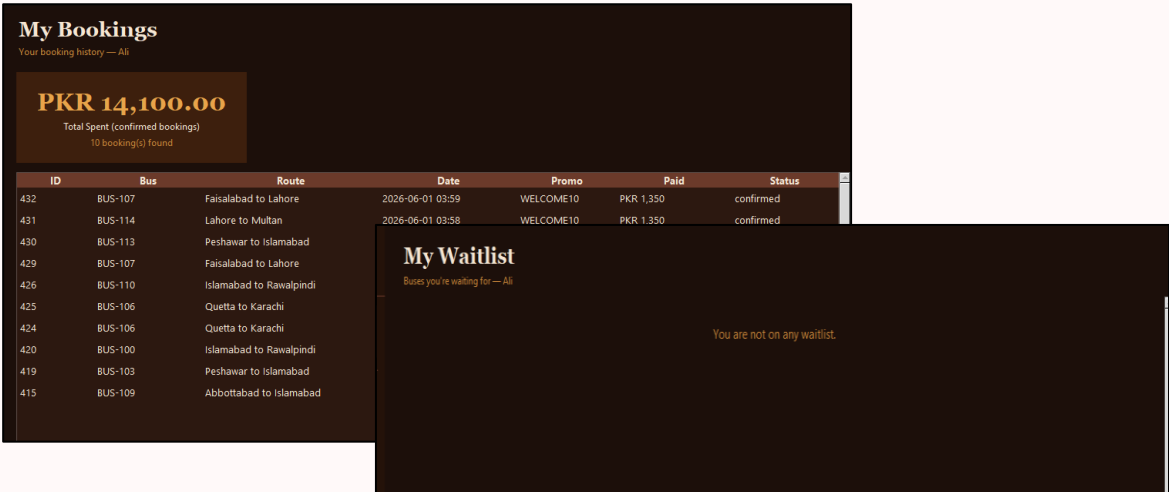
BUS-100 — Islamabad to Rawalpindi ★★★★★
nice bus
Delete

BUS-103 — Peshawar to Islamabad ★★★★★
Delete

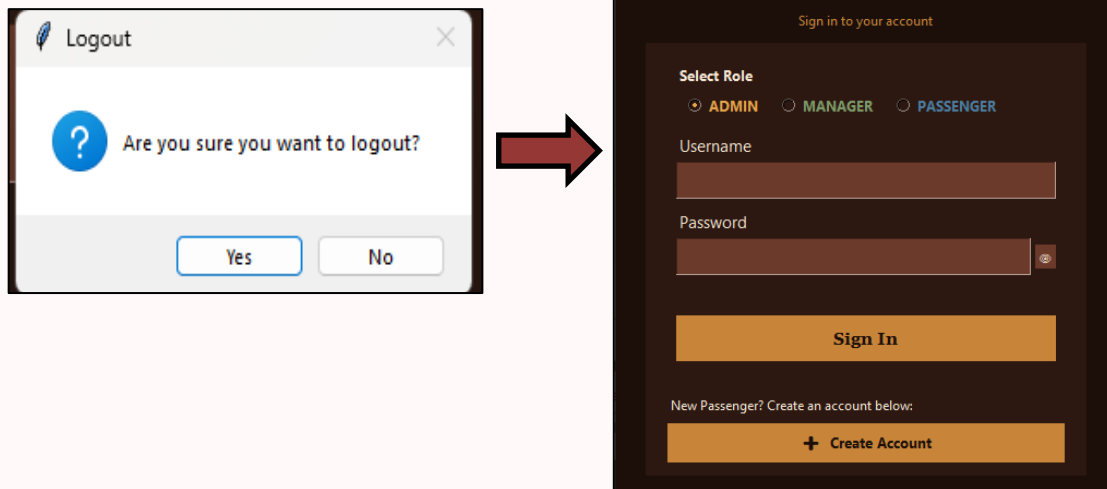
All the Payment Pop-ups that come from Book a Seat



Passenger--My bookings Panel + My Waitlist Panel



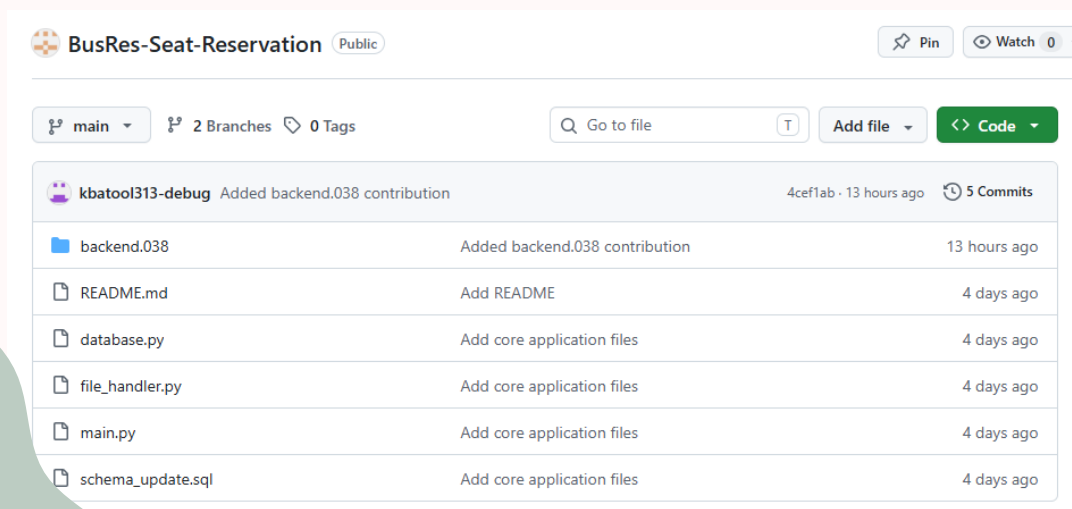
Passenger---Logout to Login Window



7.3. Literature References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*. Hoboken, NJ: Wiley, 2013.
- [3] E. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [4] Python Software Foundation, "Python 3 Documentation," 2024. [Online]. Available: <https://docs.python.org/3/>
- [5] Microsoft Corporation, "SQL Server Express Documentation," 2024. [Online]. Available: <https://learn.microsoft.com/en-us/sql/sql-server/>
- [6] Python Software Foundation, "heapq — Heap queue algorithm," 2024. [Online]. Available: <https://docs.python.org/3/library/heapq.html>
- [7] ReportLab Inc., "ReportLab PDF Library User Guide," 2024. [Online]. Available: <https://www.reportlab.com/docs/reportlab-userguide.pdf>

7.4. GitHub Contributions



7.5. Conclusion:

BusRes successfully demonstrates the practical application of Data Structures and Algorithms in a real-world, production-grade desktop system.

Beyond the mandatory requirements, BusRes implements several advanced features that demonstrate software engineering best practices: Role-Based Access Control (RBAC), cascading data integrity on bus deletion, a self-healing startup sequence, PDF ticket and invoice generation using ReportLab, a real-time waitlist system coordinated across all three user roles, case-sensitive password authentication enforced at the Python level, registered CNIC validation per user account, and an in-memory/database hybrid architecture for optimal search performance.

The system architecture cleanly separates concerns across six layers — Presentation, Business Logic, Data Access, Data, DSA, and File Export — making the codebase maintainable, extensible, and well-suited for both academic demonstration and real-world deployment.

In summary: BusRes is a 3-role desktop application built in Python with tkinter, SQL Server, and 6 DSA implementations — organized across 6 architectural layers. Every user interaction triggers a coordinated chain of method calls that synchronize between in-memory data structures and persistent database storage to deliver a consistent, validated, real-time experience.

